

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of WED 9 JULY 2025 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

Consider the following regular expression R :

$$R = a b^* a^* a \mid b a^*$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) By using the Berry-Sethi method (BS), find a deterministic finite-state automaton A that recognizes the language $L(R)$.
 - (b) Is the deterministic automaton A minimal? If not, minimize it.
 - (c) Call A_{min} the minimal automaton obtained above (point (b)). By using the Brzozowski-McCluskey method (BMC), find a regular expression R' that generates the language $L(A_{min})$.
 - (d) Find the finite-state automaton that recognizes the mirror language $L(A_{min})^R$. Is this automaton deterministic?
 - (e) Find the finite-state automaton that recognizes the complement language $\overline{L} = \Sigma^* - L(A_{min})$.
-

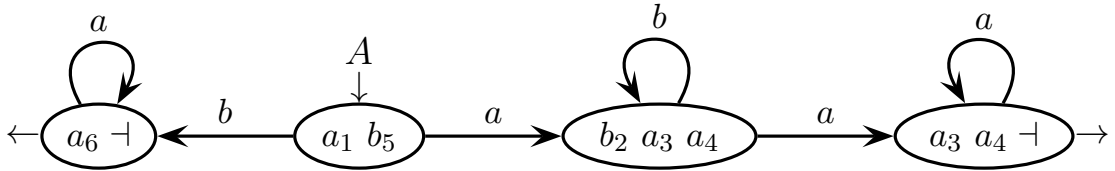
Solution

- (a) Berry-Sethi method (BS) applied to the regular expression R . First initials and followers:

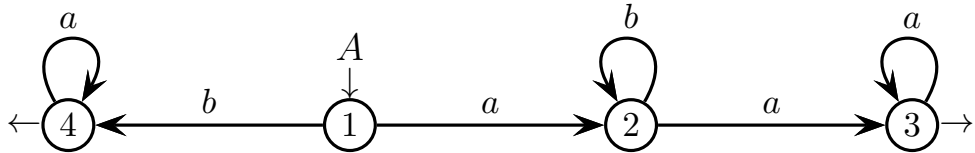
$$R = a_1 b_2^* a_3^* a_4 \mid b_5 a_6^*$$

initials	$a_1 b_5$
terminal	followers
a_1	$b_2 a_3 a_4$
b_2	$b_2 a_3 a_4$
a_3	$a_3 a_4$
a_4	$\neg$
b_5	$a_6 \neg$
a_6	$a_6 \neg$

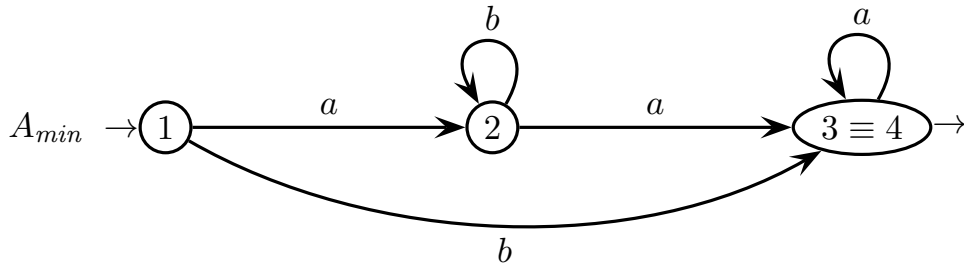
Here is the deterministic automaton A equivalent to expression R :



After renaming the nodes:

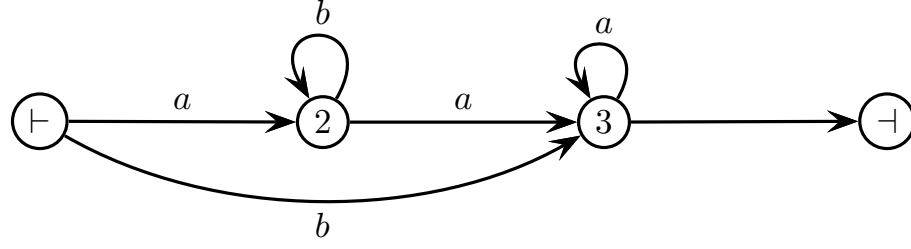


- (b) It is easily seen that the automaton A is not minimal, as the two states 3 and 4 are identical. By merging states 3 and 4, we obtain the minimal automaton A_{min} below:

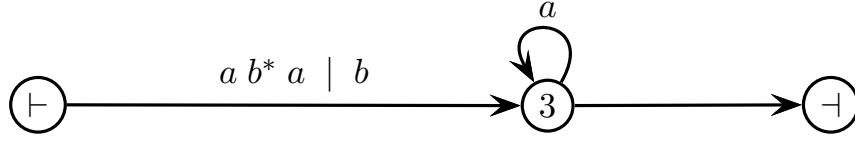


Automaton A_{min} cannot be further minimized, as the two non-final states 1 and 2 have b -transitions directed to a final and a non-final state, respectively, thus they are distinguishable, while state $3 \equiv 4$ is final. It is impossible to merge the three states.

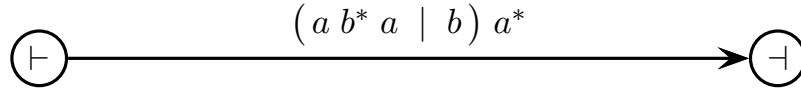
- (c) Brzozowski-McCluskey method (BMC) applied to the minimal automaton A_{min} . Normalize state $3 \equiv 4$ (renamed 3), since it has outgoing arcs (self-loop), while state 1 does not have ingoing arcs:



Eliminate state 2:



Finally eliminate state 3:

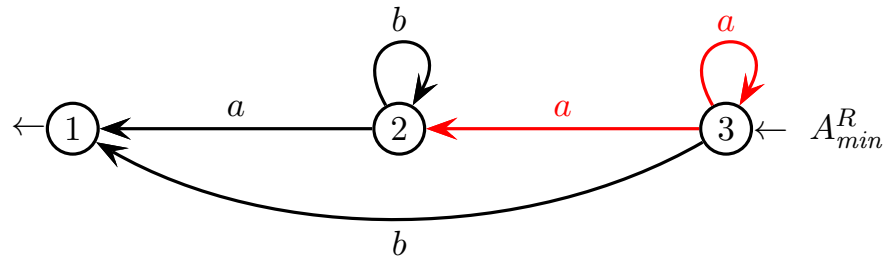


Therefore we obtain the regular expression R' below:

$$R' = (a b^* a \mid b) a^*$$

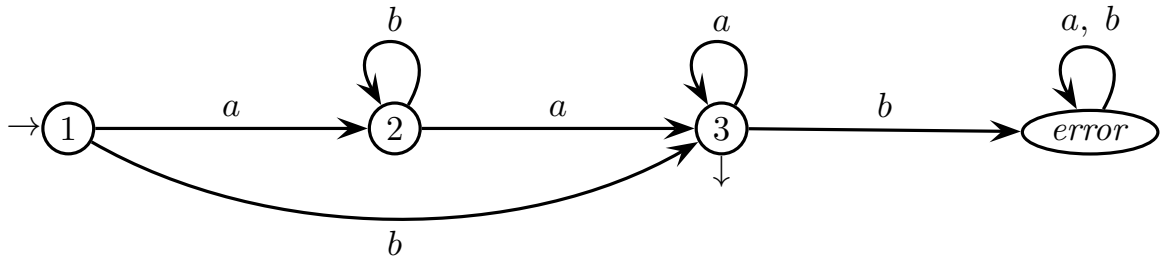
which can be easily seen to be equivalent to the original regular expression R . Moreover, if we eliminated the two states 2 and 3 in the opposite order (first 3 then 2), we would indeed (re)obtain the original expression R .

- (d) Here is the automaton A_{min}^R that recognizes the mirror language $L(A_{min})^R = L(A)^R = L(R')^R$, obtained from automaton A_{min} by reversing the arcs and switching the initial and final states:

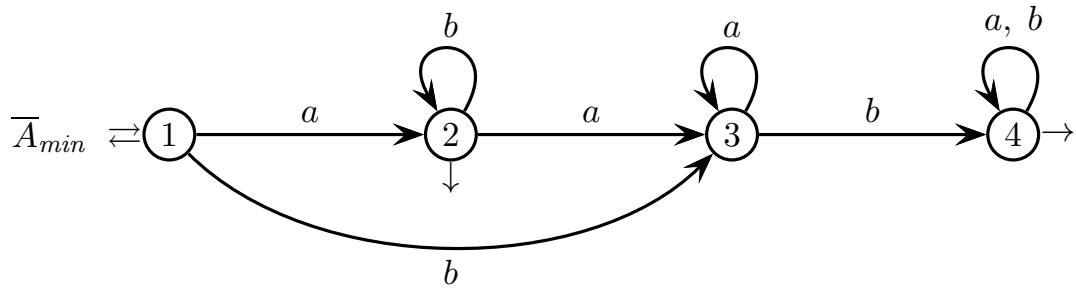


The automaton A_{min}^R is obviously non-deterministic, as shown by the red a -arcs on state 3.

- (e) Here is the automaton $\overline{A}_{min}$ that recognizes the complement language $\overline{L}(A_{min})$. It is obtained from A_{min} by first completing with an error state:



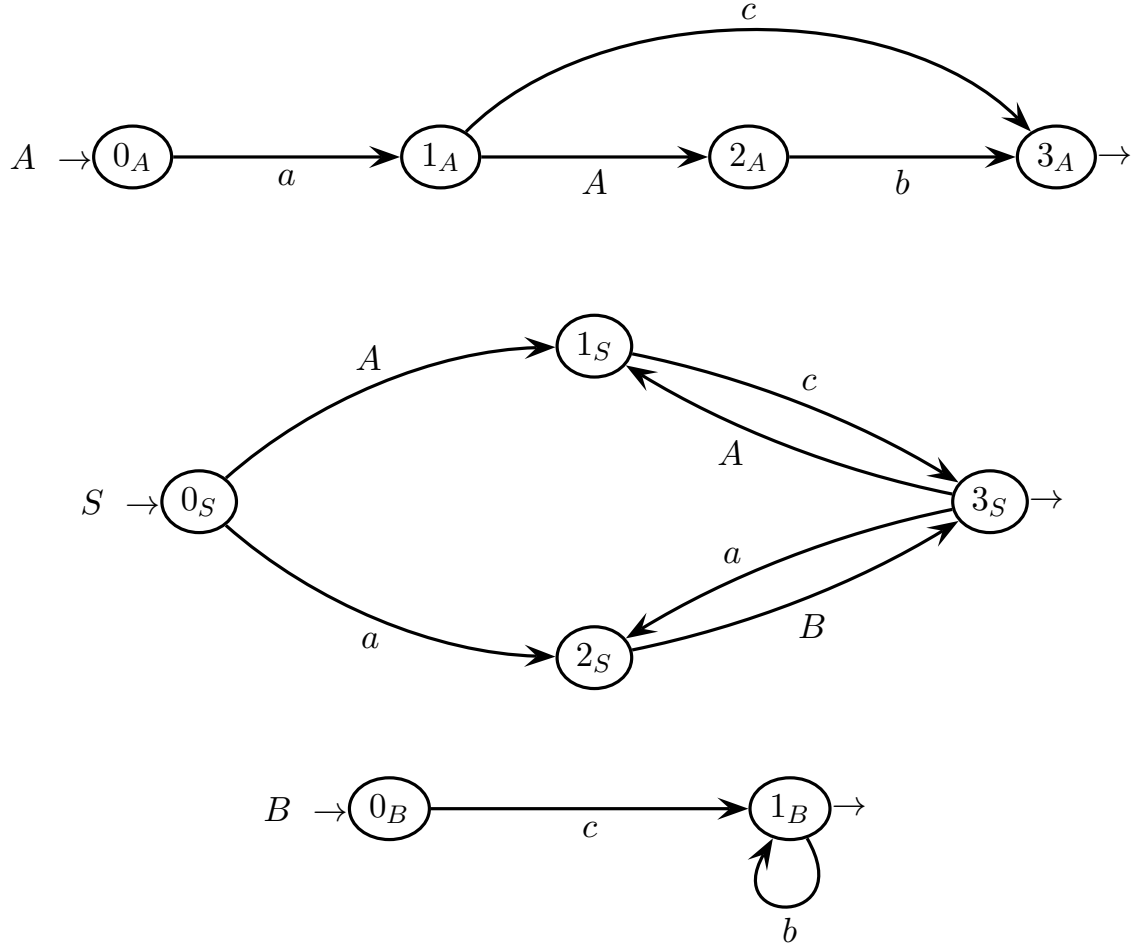
and then by switching the final and non-final states to obtain the complement automaton (for brevity the error state is renamed 4):



Automaton $\overline{A}_{min}$ is still deterministic, and it is not difficult to see that it is already minimal.

2 Syntax analysis and parsing methodologies (10 points)

Consider the EBNF grammar G below, represented as a machine network over the three-letter terminal alphabet $\Sigma = \{a, b, c\}$ and the three-symbol nonterminal alphabet $V = \{A, B, S\}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

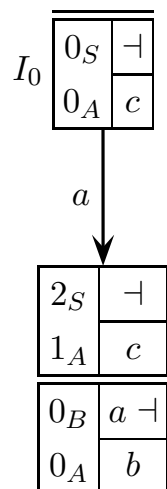
- Write all the strings of length ≤ 3 (less than or equal to three) that are generated by grammar G , and draw their syntax trees.
- Draw the complete pilot graph of grammar G and show that grammar G is ELR. Briefly justify that the pilot does not have SR, RR or Convergence conflicts.
- Simulate the recognition process of the ELR syntax analyzer of grammar G for the valid string (of length 5) below:

$$a c a c c \in L(G)$$

and show the construction process of the syntax tree of the string.

- Compute all the guide sets on the call arcs and exit arrows of the machine net of grammar G . Are there any guide set conflicts? Justify your answer.
- Say if grammar G is $ELL(k)$ for some $k \geq 1$. Justify your answer.

complete pilot of grammar G – question (b)



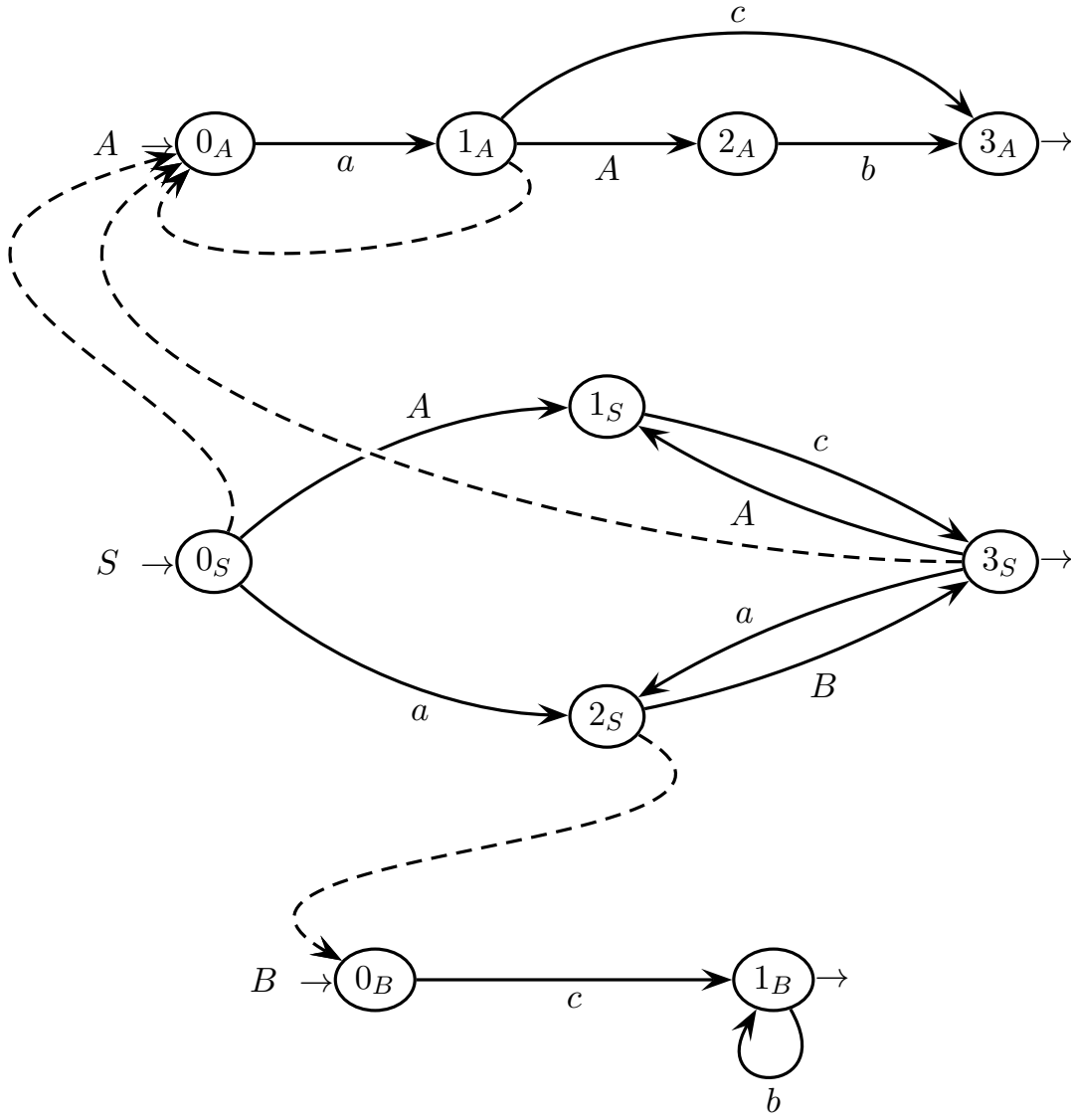
ELR conflict analysis – question (b)

simulation table of the ELR parsing process of string $a c a c c$ – question (c)
(the number of rows is not significant)

[illegible]

construction of the syntax tree – question (c)

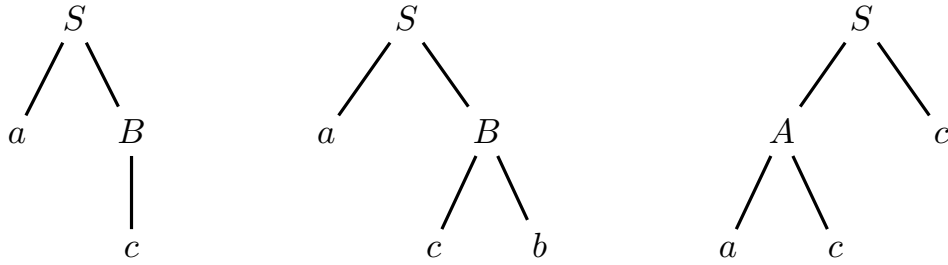
guide sets of grammar G – question (d)



ELL conflict analysis – question (d)

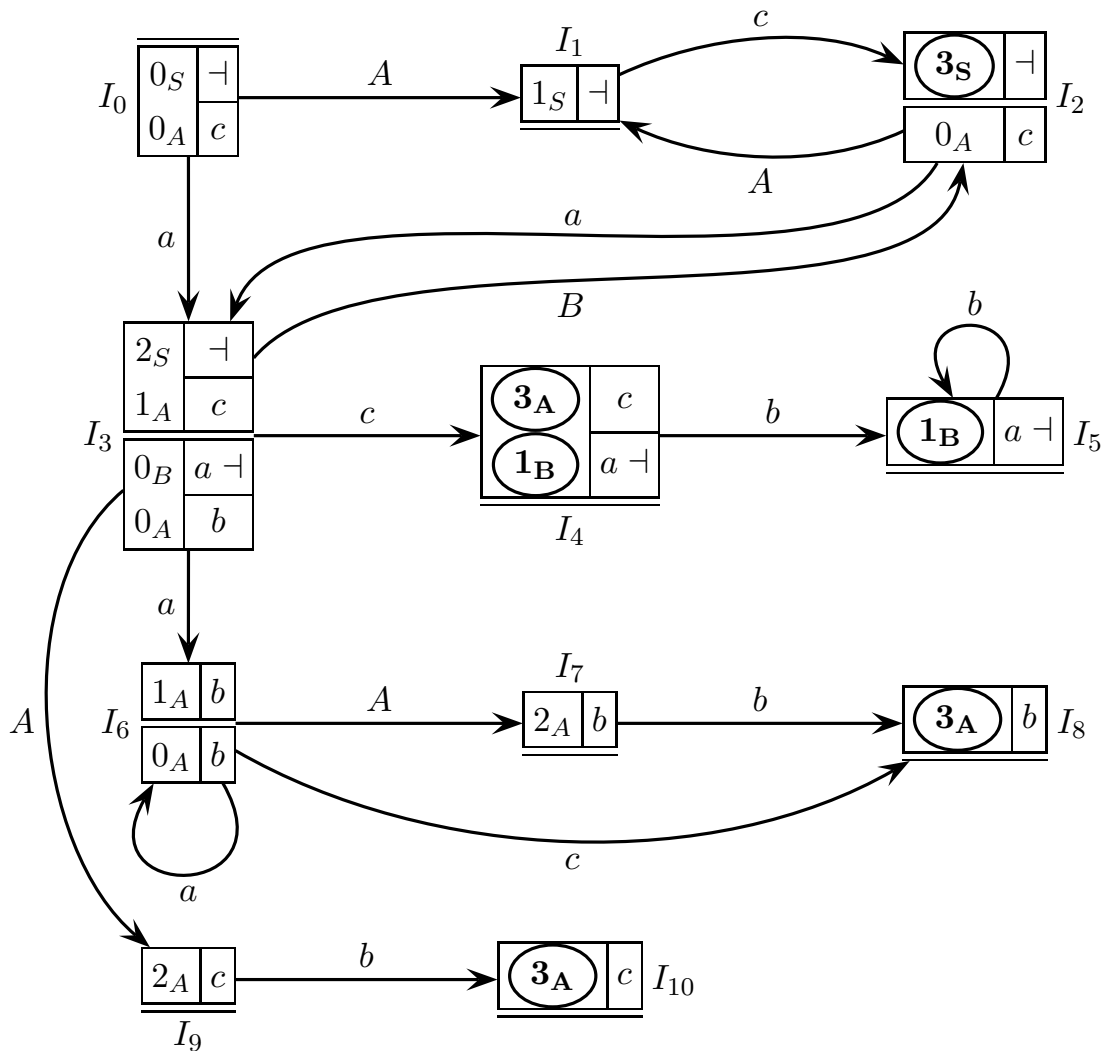
Solution

(a) Here are the valid strings of length ≤ 3 (three in total): ac , acb and acc . Their syntax trees:



Each string has exactly one tree.

(b) Here is the complete pilot of grammar G , with eleven m-states:



Grammar G is ELR. In fact, potential conflicts could only exist in these places in the pilot:

SR in the m-states I_2 , I_4 , I_5 , I_8 and I_{10} , because there are reduction (final) net states, but there are no conflicts because the look-ahead sets of the reduction candidates are disjoint from the outgoing terminal shift arcs

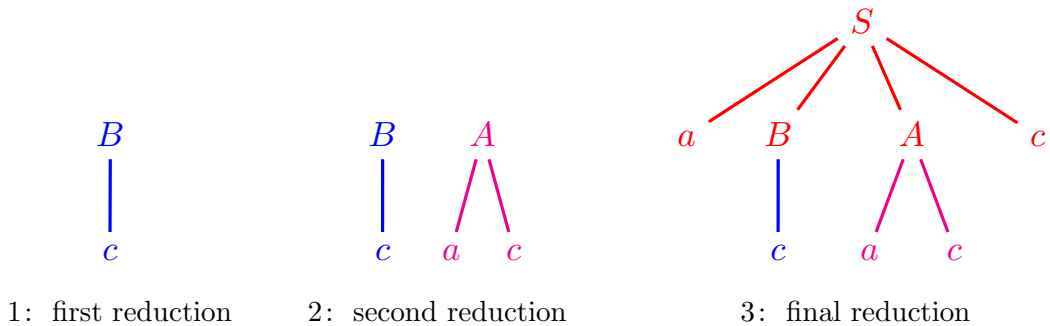
RR in the m-state I_4 , because there are two reduction (final) net states, but there are no conflicts because the look-ahead sets of the two reduction candidates are disjoint

Con in the m-states I_3 and I_4 , because they have (only) incoming multiple transitions – from I_0 and I_2 reading a for I_3 , and from I_3 reading c for I_4 – but their target net states are different, so that those transitions are not convergent: indeed the base sets for both I_3 and I_4 have two candidates, and if the transitions were convergent only one candidate would appear

In conclusion there are no conflicts of any type (SR, RR or Convergence), thus grammar G is ELR.

(c) Here is the simulation of the ELR (1) parsing process of the valid sample string $acaccc \in L(G)$:

<i>stack of m-states and grammar symbols</i>	<i>input left to scan</i>	<i>move executed</i>
I_0	$a\ c\ a\ c\ c\ \vdash$	initial configuration
$I_0\ a\ I_3$	$c\ a\ c\ c\ \vdash$	terminal shift: $I_0 \xrightarrow{a} I_3$
$I_0\ a\ I_3\ c\ I_4$	$a\ c\ c\ \vdash$	terminal shift: $I_3 \xrightarrow{c} I_4$
$I_0\ a\ I_3$	$a\ c\ c\ \vdash$	1: reduction : $c \rightsquigarrow B$
$I_0\ a\ I_3\ B\ I_2$	$a\ c\ c\ \vdash$	nonterminal shift: $I_3 \xrightarrow{B} I_2$
$I_0\ a\ I_3\ B\ I_2\ a\ I_3$	$c\ c\ \vdash$	terminal shift: $I_2 \xrightarrow{a} I_3$
$I_0\ a\ I_3\ B\ I_2\ a\ I_3\ c\ I_4$	$c\ \vdash$	terminal shift: $I_3 \xrightarrow{c} I_4$
$I_0\ a\ I_3\ B\ I_2$	$c\ \vdash$	2: reduction : $a\ c \rightsquigarrow A$
$I_0\ a\ I_3\ B\ I_2\ A\ I_1$	$c\ \vdash$	nonterminal shift: $I_2 \xrightarrow{A} I_1$
$I_0\ a\ I_3\ B\ I_2\ A\ I_1\ c\ I_2$	$\vdash$	terminal shift: $I_1 \xrightarrow{c} I_2$
I_0	$\vdash$	3: reduction : $a\ B\ A\ c \rightsquigarrow S$
initial stack	empty input	last move reduction to axiom
acceptance condition verified – see the tree construction below		

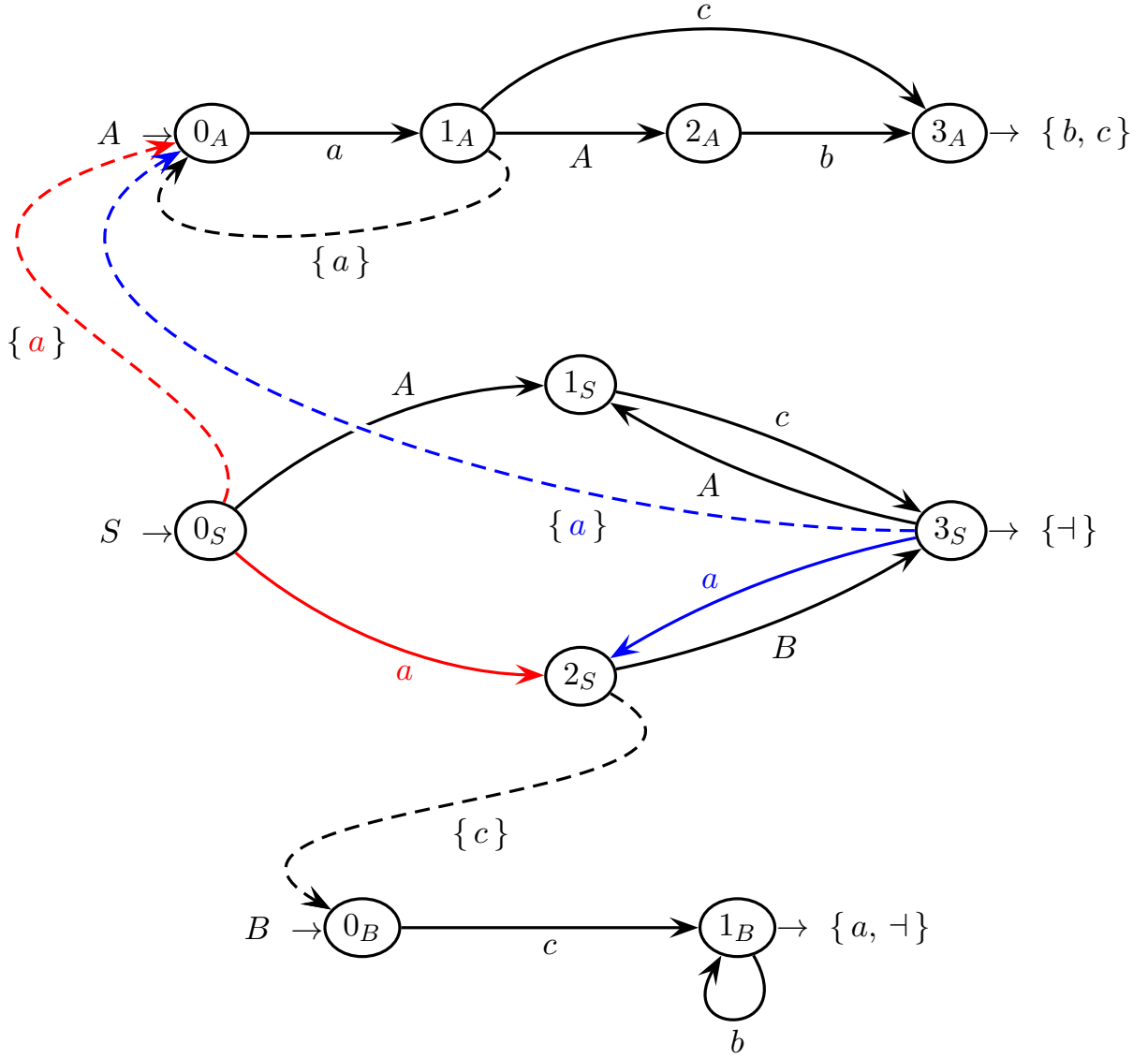


Just for completeness, here is the rightmost derivation of the string $acaccc$:

$$\begin{array}{ccccccc}
 S & \xRightarrow{3} & a\ B\ A\ c & \xRightarrow{2} & a\ B\ a\ c\ c & \xRightarrow{1} & a\ c\ a\ c\ c \\
 \uparrow & & \uparrow & & \uparrow & & \\
 & & & & & &
 \end{array}$$

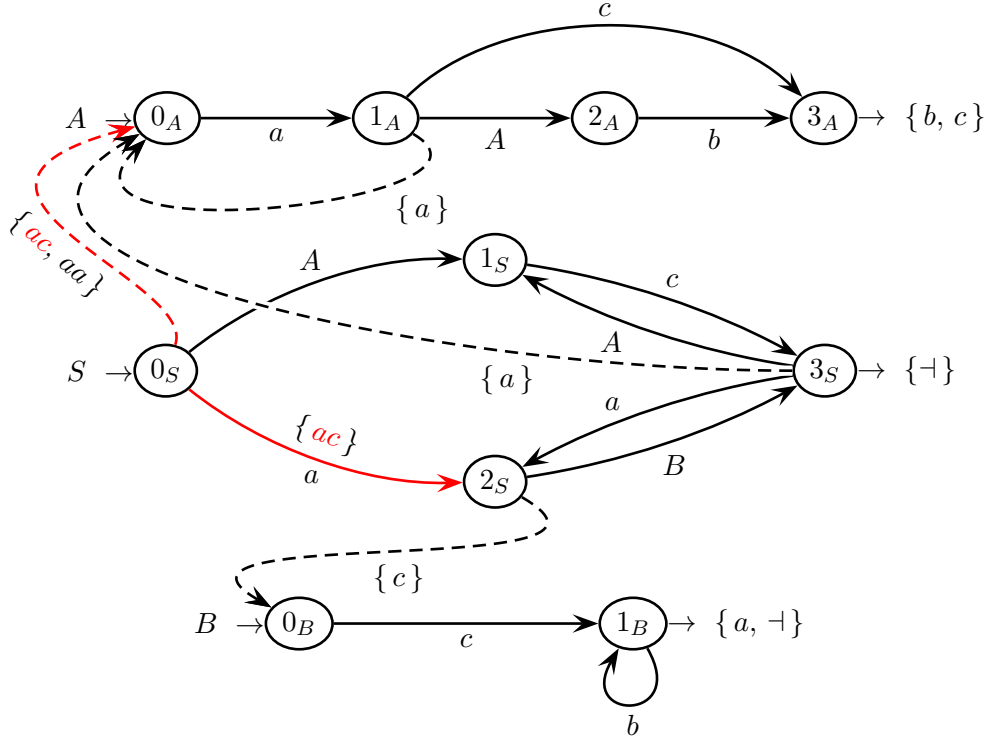
The derivation uses the above reductions 1 2 3 in reverse order, i.e., 3 2 1.

(d) Here are the guide sets (for look-ahead depth $k = 1$) on the call arcs and exit arrows:

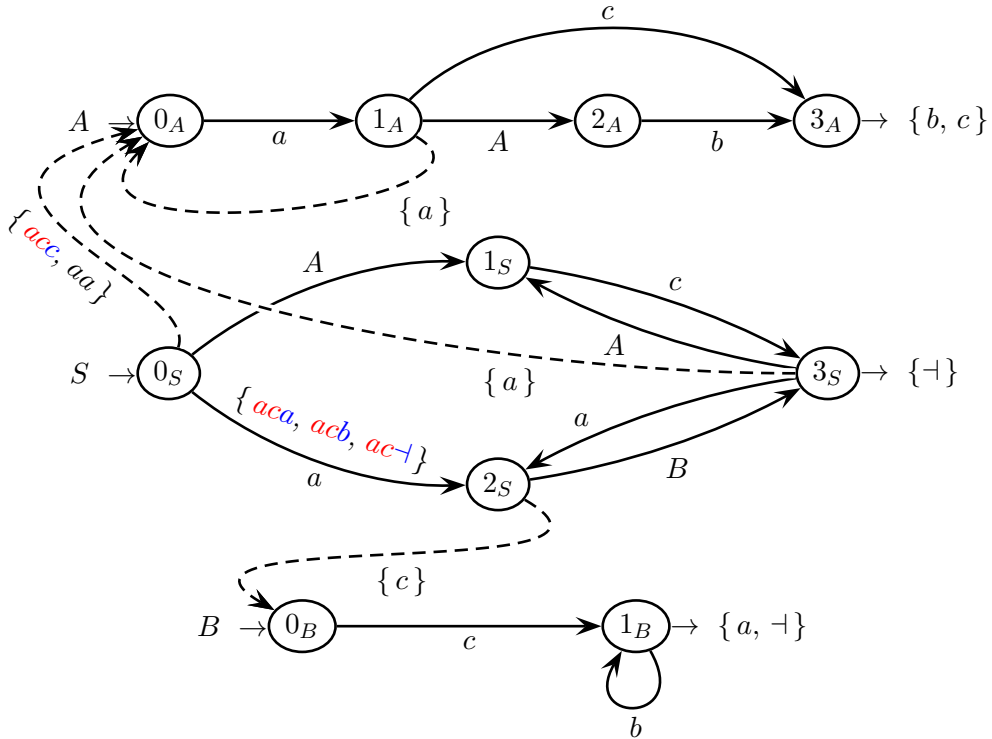


There are four divergent net states that involve call arcs or exit arrows: state 2_S with no conflicts, states 0_S and 3_S , each with a conflict on terminal a (in red and blue, respectively), and state 1_A with no conflicts. Therefore grammar G is not ELL (1), as it was expected since its pilot does not have the Single Transition Property (STP), e.g., transition $I_0 \xrightarrow{a} I_3$ is double.

- (e) It turns out that grammar G is ELL (3). For $k = 1$, the only conflicting net states are 0_S and 3_S . Their conflicts are of the same type, as both involve a call A -arc to state 0_A and a shift a -arc to state 2_S , thus it suffices to examine only one of them and increase k . Try with $k = 2$ only for 0_S :



State 0_S is still conflicting on string ac (in red). Try with $k = 3$ only for 0_S on string ac :



State 0_S is no longer conflicting. Now the two look-ahead sets are disjoint, because the third input symbol (in blue) separates their elements: character c in the set on the call arc, and characters a , b and $-$ in the set on the terminal shift arc. The same happens with state 3_S , but is not shown here. Therefore grammar G is ELL (3). Of course, grammar G is ELR (3), as for any fixed $k \geq 1$, ELR (k) includes ELL (k). Yet this does not add anything to what was already known, since from point (b) grammar G is even ELR (1), and for any fixed $k \geq 1$, ELR ($k + 1$) includes ELR (k).

3 Grammar design – translation – semantic analysis (5 + 5 points)

1. Consider a language for configuration files, consisting in a list of definitions of **configuration variables**. Configuration variables are booleans that can be turned *on* or *off*. A basic example of a configuration file is the following:

```
FAST_DEMOLITION = ON;
PLANT_TREES = OFF;
```

FAST_DEMOLITION and PLANT_TREES are examples of variable identifiers, and are symbolized by the terminal `id`. Each definition consists of the identifier `id`, followed by an equals sign “=”, the configuration value `ON` or `OFF`, and a terminating semicolon “;”. A file cannot be empty.

On the left side of an equals sign, it is also possible to specify multiple variables, separated by commas “,”, to set all of them at the same time:

```
FAST_DEMOLITION, USE_MAGIC_MALLET = ON;
USE_CHEAP_HELMETS, USE_CHEAP_RESTRAINTS, USE_CHEAP_TRUSSES = OFF;
```

On the right side of an equals sign, instead of a single literal `ON` or `OFF`, it is possible to have an arbitrary expression that combines other variables by using the logical operators *and*, *or* and *not*:

```
HIRE_SORCERER = (FAST_DEMOLITION & USE_MAGIC_MALLET) & !USE_CHEAP_HELMETS;
MAGIC_TREE_GROWTH = HIRE_SORCERER | PLANT_TREES;
```

The rules for the expressions are the following:

- the *and* operator appears as “&”, *or* appears as “|”, and *not* appears as “!”
- operator *and* is left-associative and *or* is right-associative
- operator *not* has a higher priority than *and*
- operator *and* has a higher priority than *or*
- subexpressions can be arbitrarily grouped in parentheses “()”

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Write an EBNF non-ambiguous grammar (axiom `FILE`) that generates the language described.
- (b) Draw the syntax tree of the sample configuration file shown below:

```
FAST_DEMOLITION, USE_MAGIC_MALLET = OFF;
HIRE_SORCERER = FAST_DEMOLITION & !USE_MAGIC_MALLET;
```

- (c) Extend the grammar to add the possibility of arbitrarily grouping non-empty lists of variable definitions within (possibly nested) curly brackets “{ }”:

```
PLANT_TREES = OFF;
{
    USE_CHEAP_HELMETS, USE_CHEAP_RESTRAINTS, USE_CHEAP_TRUSSES = OFF;
    {
        HIRE_SORCERER = (FAST_DEMOLITION & USE_MAGIC_MALLET) & !USE_CHEAP_HELMETS;
    }
    FAST_DEMOLITION, USE_MAGIC_MALLET = ON;
}
```

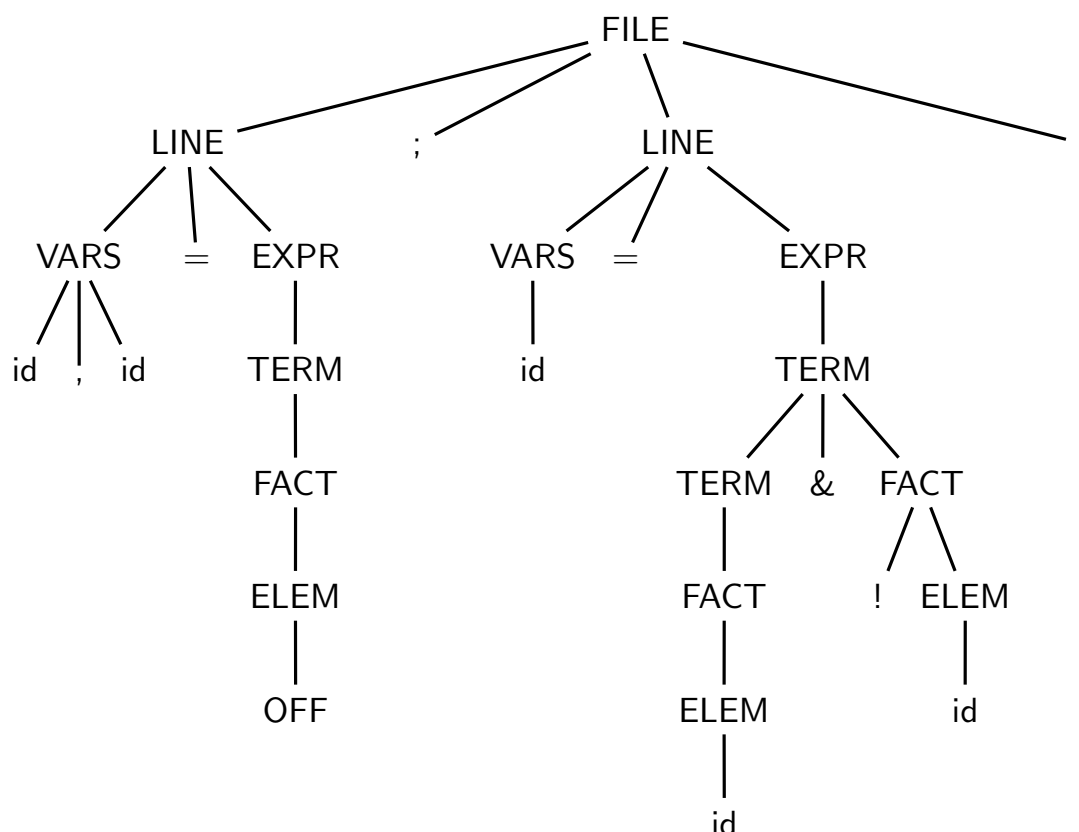
Solution

- (a) Here is a viable grammar, EBNF and non-ambiguous, for the described language (axiom FILE):

$\langle \text{FILE} \rangle \rightarrow (\langle \text{LINE} \rangle \text{ “;” })^+$	// list with term.s
$\langle \text{LINE} \rangle \rightarrow \langle \text{VARS} \rangle \text{ “=” } \langle \text{EXPR} \rangle$	// conf. var(s) def.
$\langle \text{VARS} \rangle \rightarrow \text{id } (\text{ “,” id })^*$	// list with sep.s
$\langle \text{EXPR} \rangle \rightarrow \langle \text{TERM} \rangle \text{ “ ” } \langle \text{EXPR} \rangle \mid \langle \text{TERM} \rangle$	// right-associative <i>or</i>
$\langle \text{TERM} \rangle \rightarrow \langle \text{TERM} \rangle \text{ “&” } \langle \text{FACT} \rangle \mid \langle \text{FACT} \rangle$	// left-associative <i>and</i>
$\langle \text{FACT} \rangle \rightarrow [\text{ “!” }] \langle \text{ELEM} \rangle$	// optional <i>not</i>
$\langle \text{ELEM} \rangle \rightarrow \text{ON} \mid \text{OFF} \mid \text{id} \mid \text{ “(” } \langle \text{EXPR} \rangle \text{ “)”}$	// literal, id or subexpr.

The grammar is EBNF (except where associativity is forced through BNF rules) and non-ambiguous. Operator associativity and priority are modeled according to the specifications. The grammar allows the literals ON and OFF to also appear in structured expressions, e.g., !ON and ON & id are allowed. Grammar variants that allow the literals ON and OFF to only appear in isolation on the right side of an assignment (disallow !ON, etc), are also acceptable.

- (b) Here is the syntax tree of the sample configuration file, with (in total) three variable definitions:



The various identifiers id on the tree frontier could be associated with the actual variable names.

- (c) To allow (recursive) blocks of definitions, a simple change to the rule of the (axiom) nonterminal FILE is sufficient:

$$\langle \text{FILE} \rangle \rightarrow (\langle \text{LINE} \rangle \text{ “;” } \mid \text{ “{” } } \langle \text{FILE} \rangle \text{ “} \text{”})^+ \quad // \text{ list of def.s and blocks}$$

From example (c), here we assume that a block does not have a terminating semicolon.

2. Consider the language of the expressions with a binary operator $+$, where each (sub)expression can be either of *prefix* type (Polish) or of *postfix* type (reverse Polish). Prefix-type and postfix-type subexpressions are distinguished by being enclosed in round parentheses “()” and square parentheses “[]”, respectively; the whole expression itself is enclosed in parentheses.

This language is generated by the following source BNF grammar G , over the three-symbol nonterminal alphabet $V = \{S, A, B\}$ and the six-letter terminal alphabet $\Sigma = \{+, n, “(”, “)” “[”, “]”\}$ (axiom S); see also the sample valid string aside:

$$G \left\{ \begin{array}{l} 1: S \rightarrow “(” + A A “)” \\ 2: S \rightarrow “[” B B + “]” \\ 3: S \rightarrow n \\ \hline 4: A \rightarrow + A A \\ 5: A \rightarrow n \\ 6: A \rightarrow “[” B B + “]” \\ \hline 7: B \rightarrow B B + \\ 8: B \rightarrow n \\ 9: B \rightarrow “(” + A A “)” \end{array} \right. \quad \begin{array}{l} \text{sample string of the source language } L(G) \\ \\ \\ (+ + n n [n n +]) \end{array}$$

Consider now the language of the *infix* expressions with a binary operator $+$. These expressions are specified so that *each* operator (including the top-level one) is enclosed in round parentheses. A (BNF) grammar G' that generates this language, over the one-symbol nonterminal alphabet $V' = \{S\}$ and the four-letter terminal alphabet $\Sigma' = \{+, n, “(”, “)”\}$, is (axiom S):

$$G' \left\{ \begin{array}{l} S \rightarrow “(” S + S “)” \\ S \rightarrow n \end{array} \right. \quad \begin{array}{l} \text{sample string of language } L(G') \\ ((n + n) + (n + n)) \end{array}$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Write a BNF translation grammar or scheme G^τ , with source grammar G , that syntactically translates a string of the source language $L(G)$ to the destination (target) language $L(G')$. For instance (the double brackets “[]” just delimit the expression given as argument to translation τ , and disappear from the result):

$$\tau \llbracket (+ + n n [n n +]) \rrbracket = ((n + n) + (n + n))$$

Caveat: grammar G' is *not* the destination (target) grammar of G^τ ; it only serves to clearly specify the destination (target) language of G^τ .

- (b) Draw the translation tree of grammar G^τ for the sample translation shown above (point (a)).
- (c) Consider the language of infix binary expressions that admits both round parentheses and square parentheses. Modify the previously defined translation grammar or scheme G^τ to retain the kind of parenthesis that encloses each expression in the translation (call θ the new syntactic translation and G^θ the new BNF translation grammar or scheme). For instance:

$$\theta \llbracket (+ + n n [n n +]) \rrbracket = ((n + n) + [n + n])$$

You can show only the new or modified rules.

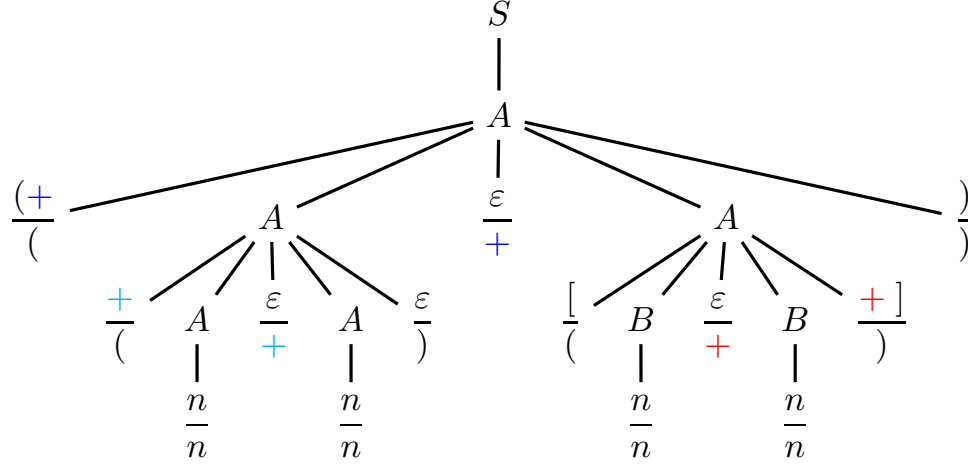
- (d) Are the translations τ and θ invertible? Explain your answer.

Solution

- (a) Here is a viable translation grammar G^τ (axiom S), which is a simple combination of the well known prefix-to-infix and postfix-to-infix transformations, with parenthesization:

$$G^\tau \left\{ \begin{array}{ll} 1: S \rightarrow \frac{“(+”}{“(”} A \frac{\varepsilon}{“+”} A \frac{“)”}{“)”} & 2: S \rightarrow \frac{“[”}{“(”} B \frac{\varepsilon}{“+”} B \frac{“+”}{“)”} \\ 3: S \rightarrow \frac{n}{n} & \\ 4: A \rightarrow \frac{“+”}{“(”} A \frac{\varepsilon}{“+”} A \frac{\varepsilon}{“)”} & 7: B \rightarrow \frac{\varepsilon}{“(”} B \frac{\varepsilon}{“+”} B \frac{“+”}{“)”} \\ 5: A \rightarrow \frac{n}{n} & 8: B \rightarrow \frac{n}{n} \\ 6: A \rightarrow \frac{“[”}{“(”} B \frac{\varepsilon}{“+”} B \frac{“+”}{“)”} & 9: B \rightarrow \frac{“(+”}{“(”} A \frac{\varepsilon}{“+”} A \frac{“)”}{“)”} \end{array} \right.$$

- (b) Translation tree for $\tau \llbracket (+ + n n [n n +]) \rrbracket = ((n + n) + (n + n))$ according to G^τ :



- (c) Here is a viable translation grammar G^θ (axiom S), with only the three rules modified from G^τ (the others are unchanged), which retain square brackets around the formerly postfix parts:

$$G^\theta \left\{ \begin{array}{l} 2: S \rightarrow \frac{“[”}{“[”} B \frac{\varepsilon}{“+”} B \frac{“+”}{“]”} \\ 6: A \rightarrow \frac{“[”}{“[”} B \frac{\varepsilon}{“+”} B \frac{“+”}{“]”} \\ 7: B \rightarrow \frac{\varepsilon}{“[”} B \frac{\varepsilon}{“+”} B \frac{“+”}{“]”} \end{array} \right.$$

- (d) Translation τ is not invertible, because the information about which subexpressions were prefix and which were postfix is lost, i.e., τ is not injective, hence not bijective. Indeed we have that:

$$\tau \llbracket (+ n n) \rrbracket = \tau \llbracket [n n +] \rrbracket = (n + n)$$

On the other hand, translation θ is invertible, because the parenthesis type encodes the type of expression that appears in the source string, e.g., $\theta \llbracket [n n +] \rrbracket = [n + n]$. Additionally, both languages have strict rules for parenthesis placement: grammar G forces a parenthesis to appear if and only if the expression type changes, while grammar G' forces a parenthesis to always appear. Therefore the parenthesization of each string in grammars G and G' is unique for all semantically-equivalent expressions and can be always reconstructed when reverse-translating.

4 Bonus question (5 points)

Consider a language L of nested lists. Each (sub)list (possibly empty) is enclosed in parentheses “()” and contains elements schematized by a letter e , without separators. The outermost list is always enclosed in parentheses. The BNF grammar G below generates language L (axiom S):

$$G \left\{ \begin{array}{l} 1: S \rightarrow (E) \\ 2: E \rightarrow \varepsilon \\ 3: E \rightarrow e E \\ 4: E \rightarrow (E) E \end{array} \right. \quad \begin{array}{l} \text{sample string of language } L = L(G) \\ (e (e e e) e ()) \end{array}$$

We want to define an attribute grammar, based on the syntactic support G , that computes a semantic translation τ . This translation transforms a nested list into a nested list with the same parenthetical structure, but without elements e . For each (sub)list:

- the opening parenthesis is immediately followed by the nesting depth (starting from the outermost list) of the contents of the (sub)list
- the closing parenthesis is immediately preceded by the total number of elements e contained (at any depth level) in the (sub)list

Here are some sample translations (the bracket pair “ $\llbracket \rrbracket$ ” just delimits the argument of τ):

$$\begin{aligned} \tau \llbracket () \rrbracket &= (1 \ 0) & \tau \llbracket (e) \rrbracket &= (1 \ 1) & \tau \llbracket (ee) \rrbracket &= (1 \ 2) & \tau \llbracket (e(e)) \rrbracket &= (1 \ (2 \ 1) \ 2) \\ \tau \llbracket (e(eee)e()) \rrbracket &= (1 \ (2 \ 3) \ (2 \ 0) \ 5) & & // \text{ sample translation for the tree} \end{aligned}$$

To this purpose, two attributes d and n (below) and their semantic functions (next pages) are given:

nonterminal	name	type	domain	meaning
E	d	right	integer ≥ 1	nesting <i>depth</i> of the contents of a (sub)list
E	n	left	integer ≥ 0	total <i>number</i> of elements e in a (sub)list

Answer the following questions (use the figures / tables / spaces on the next pages):

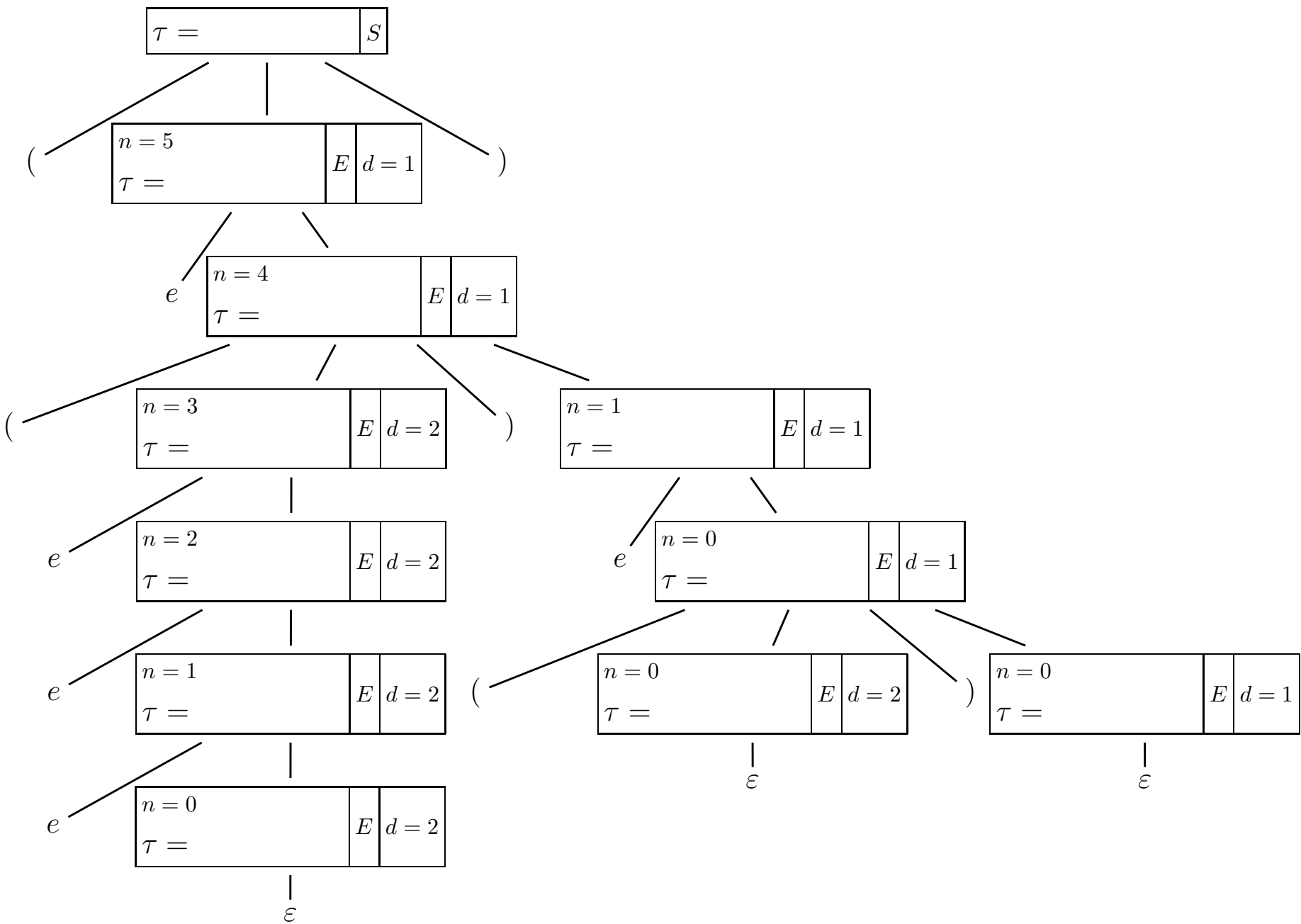
- Complete the attribute grammar already sketched (next page), by adding to it a left (synthesized) attribute τ , of value string, associated to both nonterminals S and E . The attribute τ provides, at the tree root S , the translation τ of the entire original string (you can use the operator “ $\cdot$ ” to concatenate strings and numbers). Make sure that the complete attribute grammar is one-sweep.
- Decorate the translation tree of the (longest) sample string above with the values of attribute τ .
- Is the complete attribute grammar of type L ? Explain why. If so, write the recursive-descent integrated syntactic-semantic procedure of the nonterminal S .

attribute grammar to be completed – question (a)

nonterminal	name	type	domain	meaning
S, E	τ	left	string	translation

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow (E_1)$	$d_1 := 1$ $\tau_0 :=$
2:	$E_0 \rightarrow \varepsilon$	$n_0 := 0$ $\tau_0 :=$
3:	$E_0 \rightarrow e E_1$	$d_1 := d_0$ $n_0 := n_1 + 1$ $\tau_0 :=$
4:	$E_0 \rightarrow (E_1) E_2$	$d_1 := d_0 + 1$ $d_2 := d_0$ $n_0 := n_1 + n_2$ $\tau_0 :=$

tree to be decorated with the attribute τ for the sample translation – question (b)



Solution

(a) Here is the complete attribute grammar (the semantic functions of attribute τ are in **red**):

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow (E_1)$	$d_1 := 1$ $\tau_0 := \text{"("} \cdot d_1 \cdot \tau_1 \cdot n_1 \cdot \text{"}"$ // translates the top-level list
2:	$E_0 \rightarrow \varepsilon$	$n_0 := 0$ $\tau_0 := \varepsilon$ // initializes an empty (sub)list
3:	$E_0 \rightarrow e E_1$	$d_1 := d_0$ $n_0 := n_1 + 1$ $\tau_0 := \tau_1$ // propagates the translation as-is
4:	$E_0 \rightarrow (E_1) E_2$	$d_1 := d_0 + 1$ $d_2 := d_0$ $n_0 := n_1 + n_2$ $\tau_0 := \text{"("} \cdot d_1 \cdot \tau_1 \cdot n_1 \cdot \text{"}" \cdot \tau_2$ // translates a sublist

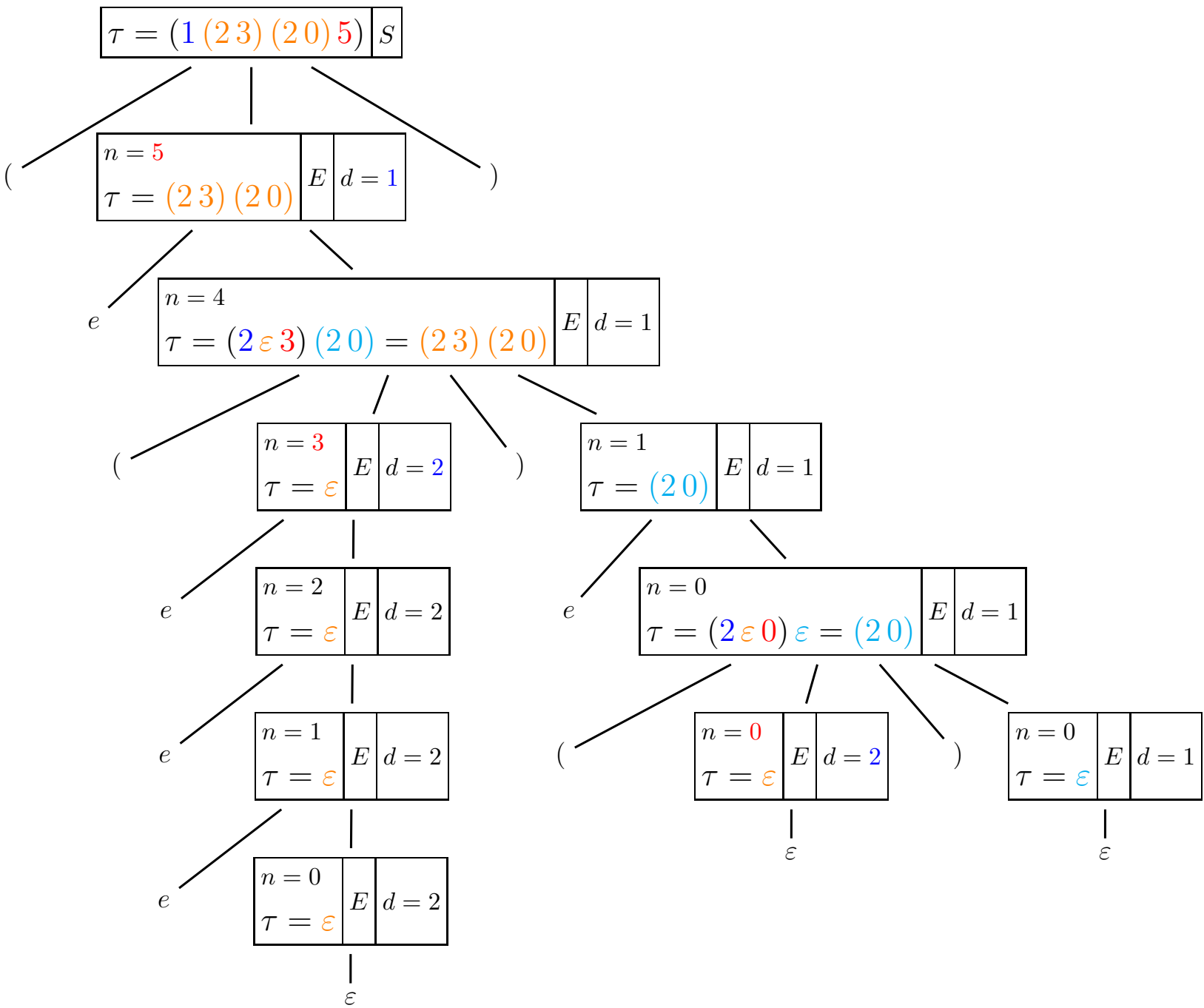
Simply put, attributes d and n compute, for each (sub)list, the depth of the contents and the total number of elements e (at any depth). Attribute τ builds the translation by retaining the parenthetic structure of the (sub)list, and by replacing the elements e with the depth d at the (sub)list start, and with the number n at the (sub)list end; see rules 2 and 4. In the rule 4, the translation of the leftmost inner sublist is inserted (with τ_1), and the one of the (sub)list contents on the right is concatenated (with τ_2). Rule 1 is similar to rule 4, but there is no concatenation (of τ_2), since the translation of the entire list is ultimately produced. Rule 2 sets to ε the translation of an empty (sub)list, and rule 3 propagates bottom-up the translation. Rule 1 sets to value 1 the depth d of the contents of the entire list, and rule 2 sets to value 0 the number n of elements of an empty (sub)list. Therefore, the algorithm for computing the translation τ , as is encoded by the above semantic functions, appears to conform to the specifications.

The attribute grammar is acyclic: attribute d depends only on itself and propagates one-way top-down, attribute n depends only on itself and propagates one-way bottom-up, so their dependency graphs are separate and acyclic; attribute τ depends on itself, and on d and n , which are already known to be acyclic, and propagates one-way bottom-up, so its dependency graph is also acyclic.

The attribute grammar is one-sweep: the only right (inherited) attribute is d and depends exclusively on itself, so the one-sweep condition is soon satisfied. For a simulation of the attribute computation, see point (b), where the coloring highlights the more relevant dependencies.

In conclusion, the attribute grammar is correct, i.e., it produces the specified translation, is computable in a general way, i.e., is acyclic, and in particular is computable in a one-sweep way.

(b) Decorated tree of the sample translation $\tau \llbracket (e(e(ee)e(e))) \rrbracket = (1(23)(20)5)$:



- (c) Grammar G is of type LL (1), as the guide sets for the alternative rules 2, 3 and 4 of nonterminal E are disjoint: for 2 it is $\{ ") " \}$, for 3 it is $\{ e \}$ and for 4 it is $\{ " (" \}$; the last holds also for the axiom S . There are no dependencies between the attributes of sibling nodes, thus the sibling graphs (of all rules) are empty and do not interfere with the LL parsing order. Since (i) the syntactic support G is LL, (ii) the attribute grammar is one-sweep (by design), and (iii) the evaluation order of sibling nodes (for each rule) coincides with the LL parsing order, then the L condition is satisfied.

Therefore the attribute grammar has a recursive-descent integrated syntactic-semantic analyzer. Here is a viable procedure for nonterminal S , with all the error cases grouped at the end:

```

procedure  $S$  (out  $\tau_0$ : string)           //  $\tau_0$  will contain the entire translation
local vars   $d_1, n_1$ : integer   $\tau_1$ : string           // axiomatic rule  $S \rightarrow ( E )$ 
if  $cc \in \{ " ( " \}$  then                               // test for (
     $cc := next$                                            // shift on (
    if  $cc \in \{ e, " ( ", " ) " \}$  then                 // test for e, ( or )
         $d_1 := 1$                                          // assign  $d_1$ 
        call  $E$  (in  $d_1$ ; out  $n_1, \tau_1$ )             // input  $d_1$ , output  $n_1$  and  $\tau_1$ 
        if  $cc \in \{ " ) " \}$  then                       // test for )
             $cc := next$                                    // shift on )
            if  $cc \in \{ \neg \}$  then                     // test for input end
                 $\tau_0 := " ( " \cdot d_1 \cdot \tau_1 \cdot n_1 \cdot " ) "$  // assign  $\tau_0$ 
                return                                   // return  $\tau_0$ 
            error                                         // all error cases
        error
    error
end procedure

```

This quickly concludes the procedure of nonterminal S . For a better understanding, here is a version of the procedure encoded in a literal way, with separated error cases:

```

procedure  $S$  (out  $\tau_0$ : string)           //  $\tau_0$  will contain the entire translation
local vars   $d_1, n_1$ : integer   $\tau_1$ : string           // axiomatic rule  $S \rightarrow ( E )$ 
if  $cc \in \{ " ( " \}$  then                               // test for (
     $cc := next$                                            // shift on (
    if  $cc \in \{ e, " ( ", " ) " \}$  then                 // test for e, ( or )
         $d_1 := 1$                                          // assign  $d_1$ 
        call  $E$  (in  $d_1$ ; out  $n_1, \tau_1$ )             // input  $d_1$ , output  $n_1$  and  $\tau_1$ 
        if  $cc \in \{ " ) " \}$  then                       // test for )
             $cc := next$                                    // shift on )
            if  $cc \in \{ \neg \}$  then                     // test for input end
                 $\tau_0 := " ( " \cdot d_1 \cdot \tau_1 \cdot n_1 \cdot " ) "$  // assign  $\tau_0$ 
                return                                   // return  $\tau_0$ 
            else error                                   // expected input end, but found character  $cc$ 
        else error                                       // expected character ), but found character  $cc$ 
    else error                                           // expected character e, ( or ), but found character  $cc$ 
else error                                             // expected character (, but found character  $cc$ 
end procedure

```

With the latter version, one could provide an accurate error diagnostic; see the comments aside, which sketch possible error messages, e.g., in C language, `fprintf (stderr, "expected input end, but found character %c", cc)`, and similarly for the others.

Just for completeness, optimizations of local variables are possible. For instance, basing of the initialization $d_1 := 1$, variable d_1 could be removed, as it is not modified any more subsequently:

```

procedure  $S$  (out  $\tau_0$ : string)           //  $\tau_0$  will contain the entire translation
local vars    $n_1$ : integer    $\tau_1$ : string           // axiomatic rule  $S \rightarrow (E)$ 
if  $cc \in \{“(”\}$  then                               // test for (
     $cc := next$                                        // shift on (
    if  $cc \in \{e, “(”, “)”\}$  then                     // test for e, ( or )
        call  $E$  (in 1; out  $n_1, \tau_1$ )           // replace  $d_1$  by 1, output  $n_1$  and  $\tau_1$ 
        if  $cc \in \{“)”\}$  then                         // test for )
             $cc := next$                                // shift on )
            if  $cc \in \{\neg\}$  then                     // test for input end
                 $\tau_0 := “(” \cdot 1 \cdot \tau_1 \cdot n_1 \cdot “)”$  // replace  $d_1$  by 1 and assign  $\tau_0$ 
                return                               // return  $\tau_0$ 
    error                                           // all error cases
end procedure

```

Furthermore, if we treat the output parameter τ_0 as a generic local variable, variable τ_1 could be removed as well by using τ_0 to temporarily store it instead:

```

procedure  $S$  (out  $\tau_0$ : string)           //  $\tau_0$  will contain the entire translation
local vars    $n_1$ : integer           // axiomatic rule  $S \rightarrow (E)$ 
if  $cc \in \{“(”\}$  then                               // test for (
     $cc := next$                                        // shift on (
    if  $cc \in \{e, “(”, “)”\}$  then                     // test for e, ( or )
        call  $E$  (in 1; out  $n_1, \tau_0$ ) // replace  $\tau_1$  by  $\tau_0$ , input 1, output  $n_1$  and  $\tau_0$ 
        if  $cc \in \{“)”\}$  then                         // test for )
             $cc := next$                                // shift on )
            if  $cc \in \{\neg\}$  then                     // test for input end
                 $\tau_0 := “(” \cdot 1 \cdot \tau_0 \cdot n_1 \cdot “)”$  // replace  $\tau_1$  by  $\tau_0$  and reassign  $\tau_0$ 
                return                               // return  $\tau_0$ 
    error                                           // all error cases
end procedure

```

Finally, again for completeness, here are the main program ANALYZER:

```

program ANALYZER           // recursive-descent syntactic-semantic analyzer
global vars    $cc$ : char    $\tau$ : string           // parse and translate the entire input
 $cc := next$                                        // initialize current char
call  $S$  (out  $\tau$ )           // invoke axiom and return translation
if  $cc \in \{\neg\}$  then accept else reject       // acceptance / rejection condition
end program

```

and the procedure for nonterminal E (for brevity, directly with error grouping):

```

//  $n_0$  and  $\tau_0$  will contain n. of elem.s  $e$  and translation of a (sub)list
procedure  $E$  (in  $d_0$ : integer; out  $n_0$ : integer,  $\tau_0$ : string) // all alternatives
local vars  $d_1, d_2, n_1, n_2$ : integer  $\tau_1, \tau_2$ : string
// alternative rule  $E \rightarrow \varepsilon$ 
if  $cc \in \{ " " \}$  then // test for )
|  $n_0 := 0$  // assign  $n_0$ 
|  $\tau_0 := \varepsilon$  // assign  $\tau_0$ 
| return // return  $n_0$  and  $\tau_0$ 
// alternative rule  $E \rightarrow e E$ 
else if  $cc \in \{ e \}$  then // test for  $e$ 
|  $cc := next$  // shift on  $e$ 
| if  $cc \in \{ e, "(", ")" \}$  then // test for  $e, ($  or  $)$ 
| |  $d_1 := d_0$  // assign  $d_1$ 
| | call  $E$  (in  $d_1$ ; out  $n_1, \tau_1$ ) // input  $d_1$ , output  $n_1$  and  $\tau_1$ 
| | if  $cc \in \{ " " \}$  then // test for )
| | |  $n_0 := n_1 + 1$  // assign  $n_0$ 
| | |  $\tau_0 := \tau_1$  // assign  $\tau_0$ 
| | | return // return  $n_0$  and  $\tau_0$ 
// alternative rule  $E \rightarrow ( E ) E$ 
else if  $cc \in \{ "(" \}$  then // test for (
|  $cc := next$  // shift on (
| if  $cc \in \{ e, "(", ")" \}$  then // test for  $e, ($  or  $)$ 
| |  $d_1 := d_0 + 1$  // assign  $d_1$ 
| | call  $E$  (in  $d_1$ ; out  $n_1, \tau_1$ ) // input  $d_1$ , output  $n_1$  and  $\tau_1$ 
| | if  $cc \in \{ " " \}$  then // test for )
| | |  $cc := next$  // shift on )
| | | if  $cc \in \{ e, "(", ")" \}$  then // test for  $e, ($  or  $)$ 
| | | |  $d_2 := d_0$  // assign  $d_2$ 
| | | | call  $E$  (in  $d_2$ ; out  $n_2, \tau_2$ ) // input  $d_2$ , output  $n_2$  and  $\tau_2$ 
| | | | if  $cc \in \{ " " \}$  then // test for )
| | | | |  $n_0 := n_1 + n_2$  // assign  $n_0$ 
| | | | |  $\tau_0 := "(" \cdot d_1 \cdot \tau_1 \cdot n_1 \cdot ")" \cdot \tau_2$  // assign  $\tau_0$ 
| | | | | return // return  $n_0$  and  $\tau_0$ 
error // all error cases
end procedure

```

Optimizations of local variables are once again possible, as before (but are not shown here).